

ARM 7 Microcontroller Real World Interfacing

3.1 Debuggers

The main concern in microcontroller programming is program memory space. Assembly language programming produces a hex file that is much smaller than that produced by programming in C.

But programming in C, has following advantages.

1. Programming in C is less tedious and less time consuming as compared to assembly.
2. It is easier to modify and update.
3. It is portable to other microcontrollers with little modification.
4. C also provides some library functions.

When we write a program in 'C' we need a compiler to convert it into the machine language. Here we will be using Pentium processor based compiler to do this programming. The compiler has to make a code of LPC2148 processor from the C program given to it such a compiler is called as cross compiler i.e. it produces the machine code for another processes, and not as that used in the system. The widely used cross compiler for LPC2148 and many other microcontrollers is "Keil μ Vision".

Keil μ vision has a header file for every microcontroller. This header file contains variable declaration, macro definitions and definitions for special function registers (SFRs). The header file for LPC21xx is LPC21xx.h or LPC214x for LPC2148. This file has given names for all the SFRs and hence it becomes convenient for the programmer to write the program directly by using these names.

3.2 Interfacing ADC, DAC and Sensors

† 3.2.1 LPC 2148 on-chip ADC

† The LPC2141/42 contain one and the LPC2144/46/48 contain two analog to digital converters. These converters are single 10-bit successive approximation analog to digital converters. While ADC0 has six channels, ADC1 has eight channels. Therefore, total number of available ADC inputs for LPC2141/42 is 6 and for LPC2144/46/48 is 14.

† Courtesy NXP/Philips

3.2.2 † Feature

- 10 bit successive approximation analog to digital converter.
- Measurement range of 0 V to VREF (2.5 V VREF VDDA).
- Each converter capable of performing more than 400000 10-bit samples per second.
- Every analog input has a dedicated result register to reduce interrupt overhead.
- Burst conversion mode for single or multiple inputs.
- Optional conversion on transition on input pin or timer match signal.
- Global Start command for both converters (LPC2142/44/46/48 only).

3.3 † LPC2148 on-chip DAC

The DAC enables the LPC2141/42/44/46/48 to generate a variable analog output. The maximum DAC output voltage is the VREF voltage.

3.3.1 Features

- 10-bit DAC.
- Buffered output.
- Power-down mode available.
- Most of the physical quantities such as temperature, pressure, displacement, vibrations etc. are available in analog form. These quantities are represented accurately in analog form but it is difficult to process, store or transmit the analog signal because noise easily introduces error.
- Hence to reduce these errors it is always better to express these physical quantities in the digital form.
- The digital representation of a signal makes storage possible, processing simpler and transmission easier.
- Most of the physical quantities such as temperature, pressure, displacement, vibrations etc. are available in analog form. These quantities are represented accurately in analog form but it is difficult to process, store or transmit the analog signal because noise easily introduces error.
- Hence to reduce these errors it is always better to express these physical quantities in the digital form.
- The digital representation of a signal makes storage possible, processing simpler and transmission easier.
- Therefore A to D conversion is necessary. Now once the processing, transmission etc. is done the signal should be brought back to its analog form, for which the D to A conversion is essential.
- Both ADC and DAC circuits are called as data converters and they are available in the IC form.
- Fig. 3.3.1 shows a A/D and D/A converter application.

† Courtesy NXP/Philips

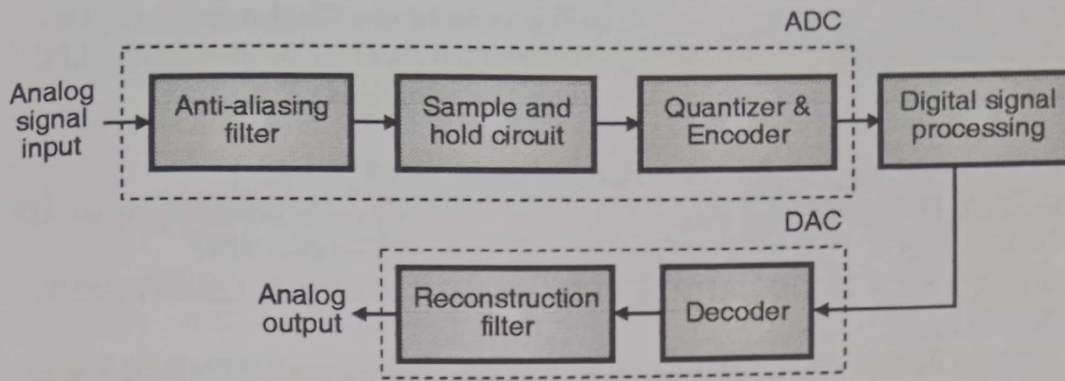


Fig. 3.3.1 : A/D and D/A converter application

As shown in the Fig. 3.3.1 the A/D conversion involves band limiting the signal, sampling the signal, quantizing it and encoding it into a suitable digital format before transmission. Once the signal is transmitted, it is received and converted back to analog form by the decoder and the reconstruction filter.

3.3.2 Interfacing ADCs using Interrupt VIC

PU - May 2015

LPC2148 has two on-chip ADCs. We will make use of the ADC0 and convert the analog signal given on its channel 3, to its equivalent digital value. In the following program, we will connect a potentiometer to the analog input of the ADC and convert it to its digital equivalent value.

Program 3.3.1 : Write a program to interface a potentiometer to the LPC2148 and measure the analog voltage and convert to its digital value.

Soln. :

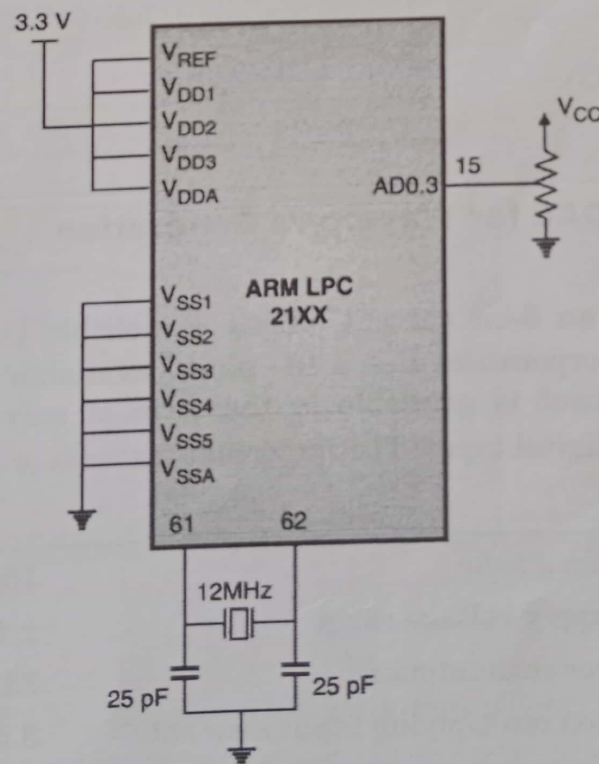


Fig. P. 3.3.1



```

#include "lpc214x.h" //This is the header file that contains all the
                    //information about the microcontroller LPC2148
void InitADC()      //Function to initialize ADC
{
    PCB_PINSEL1 = (PCB_PINSEL1 & ~PCB_PINSEL1_P030_MASK) |
    PCB_PINSEL1_P030_AD03; // Enable power supply for ADC0
    SCB_PCONP |= SCB_PCONP_PCAD0; // Initialise ADC
    AD0_CR = AD_CR_CLKS10 | AD_CR_PDN | ((3 - 1) << AD_CR_CLKDIVSHIFT)
    | AD_CR_SEL3;
    // Enable 10-bit precision ADC, Exit power-down mode 4.0MHz & Use channel 3
}
int ReadADC()      // Function to read the digital equivalent value from ADC
{
    AD0_CR &= ~(AD_CR_START_MASK | AD_CR_SELMASK);
    //Stop all conversions and deselect all channels
    {
        AD0_CR |= (AD_CR_START_NONE | AD_CR_SEL3); // Select channel 3 of ADC0
        AD0_CR |= AD_CR_START_NOW; //Start conversion
        while (!(AD0_DR3 & AD_DR_DONE)); // Wait for the completion of the conversion
        return ((AD0_DR3 & AD_DR_RESULTMASK) >> AD_DR_RESULTSHIFT);
        // Return the digital result read from ADC
    }
}
void main()
{
    InitADC(); // call the function to initialise ADC 0, Channel 3
    int results = 0; // Initialize the result to zero
    while (1) //Always true infinite loop to read the ADC digital data
    {
        results = ReadADC();
    }
}

```

3.3.3 Interfacing DAC for Waveform Generation

DAC 0808

The DAC 0808 is an 8-bit current output monolithic DAC manufactured by the National semiconductor corporation. It is a 16 - pin IC available in dual in line DIP plastic package. The analog output is available in the form of current I_o . That means I_o is proportional to the 8-bit digital input. The important features of DAC 0808 are as follows :

Features of DAC 0808

Fast settling time	150 nsec. typically
Power supply voltage range	± 4.5 mW at ± 5 V
Low power consumption.	33 mW at ± 5 V.
High speed multiplying input slew rate	8 mA / μ sec.
Interfaces directly with TTL, DTL and CMOS logic levels.	



pin configuration and functional block diagram

- The pin configuration and functional block diagram of DAC 0808 are as shown in Figs. 3.3.2(a) and (b) respectively.
- The internal block diagram shows that DAC 0808 consists of R-2R ladder along with current switches and reference current amplifier.
- A_1 to A_8 are the 8-digital input lines with A_1 as the most significant bit and A_8 as the least significant bit.
- The analog output is available in the form of current I_o , therefore we need to use an external current to voltage converter if the analog output in the form of voltage is required.
- DAC 0808 requires a dual polarity ($\pm$) supply voltage, typically $\pm 15V$, for its operation. The reference voltage can be either positive or negative.
- An external reference voltage should be applied to either $V_{REF} (+)$ or $V_{REF} (-)$ depending on the polarity of reference voltage.

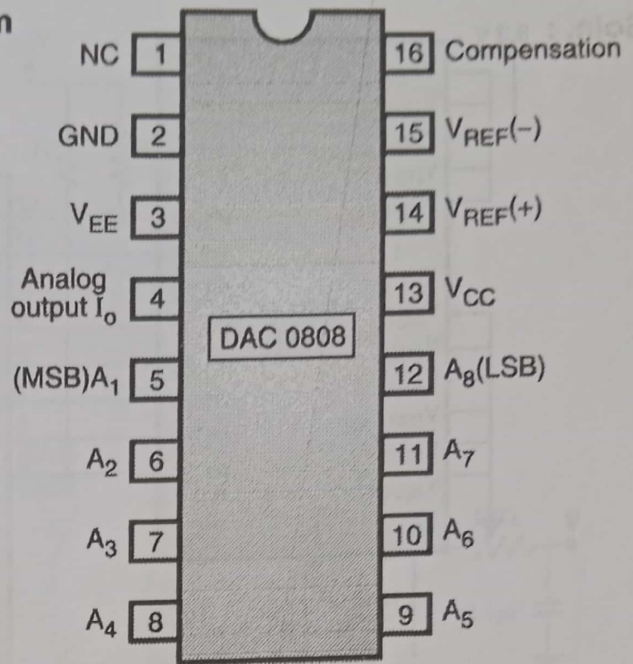


Fig. 3.3.2(a) : Pin configuration of DAC 0808

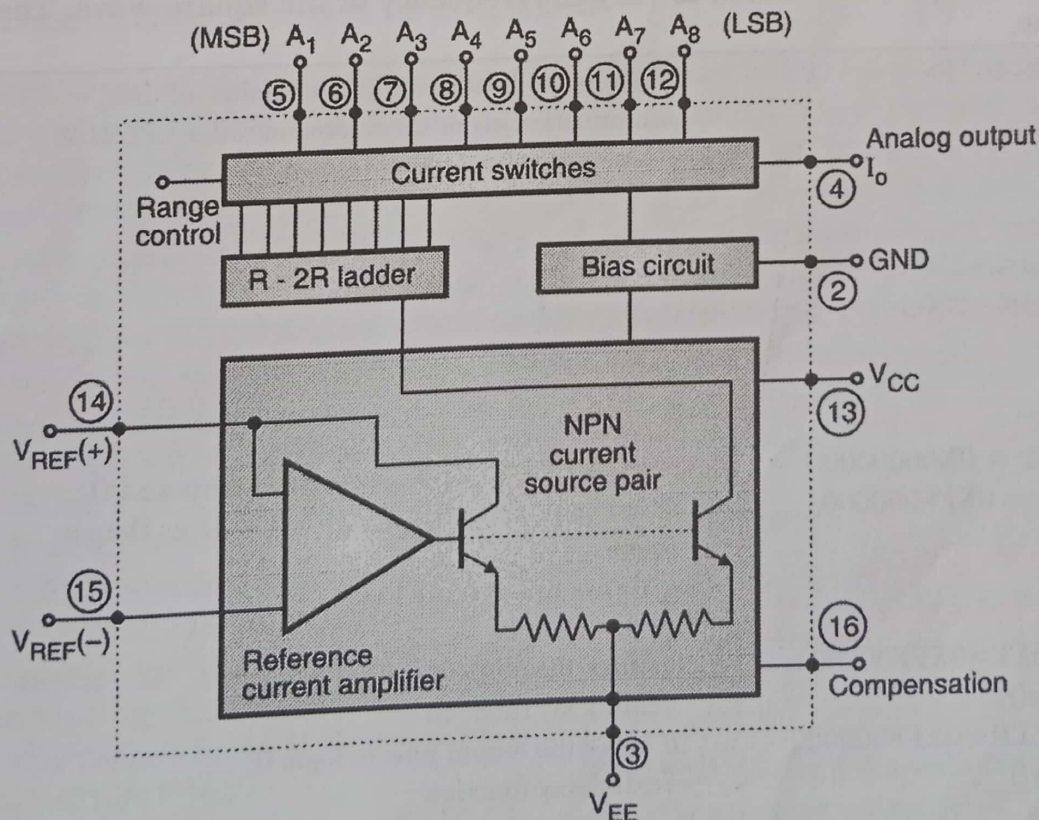


Fig. 3.3.2(b) : Functional block diagram of DAC 0808

Program 3.3.2 : Write a program to generate a square wave and show the interfacing of a DAC 0808 with LPC2148 Fig. P. 3.3.2 shows the interfacing of LPC2148 with an DAC0808.

Soln. : 3.3 V

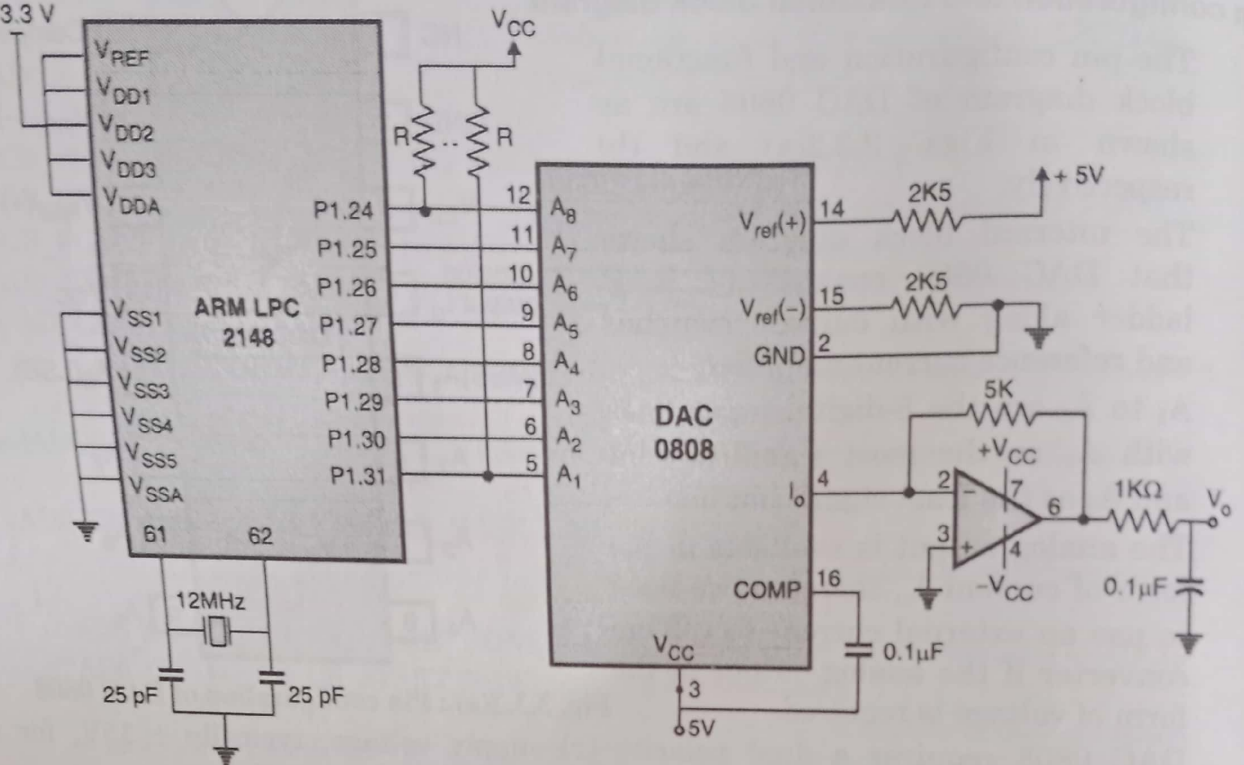


Fig. P. 3.3.2

The waveform generated will be seen on the V_o pin of the OPAMP across the capacitor. The delay can be varied to vary the frequency of the square wave. The program is given below.

```
#include <lpc2148.H>                                     //This is the header file that contains all the
                                                         //information about the microcontroller LPC2148
                                                         // Delay function

void delay()
{
    int i, j;
    for(i=0; i<=250; i++)
        for(j=0; j<=250; j++) ;
}

void main()
{
    PINSEL2 = 0X00000000;                                //Set P1.24 To P1.31 as General purpose I/O
    IO1DIR = 0XFF000000;                                  //Configure Port pins P1.24 to P1.31 as Output
                                                         //pins
    while(1)                                              //Always true infinite loop
    {
        IO1SET=0XFF000000;                                // Make the output pins to logic 1
        delay();                                           //call delay function
        IO1CLR=0XFF000000;                                // Make the output pins to logic 0
        delay() ;                                          //call delay function
    }
}
```

Program 3.3.3 : Write a program to generate a ramp wave and show the interfacing of a DAC 0808 with LPC2148 Fig. P. 3.3.3 shows the interfacing of LPC2148 with an DAC0808.

Soln. :

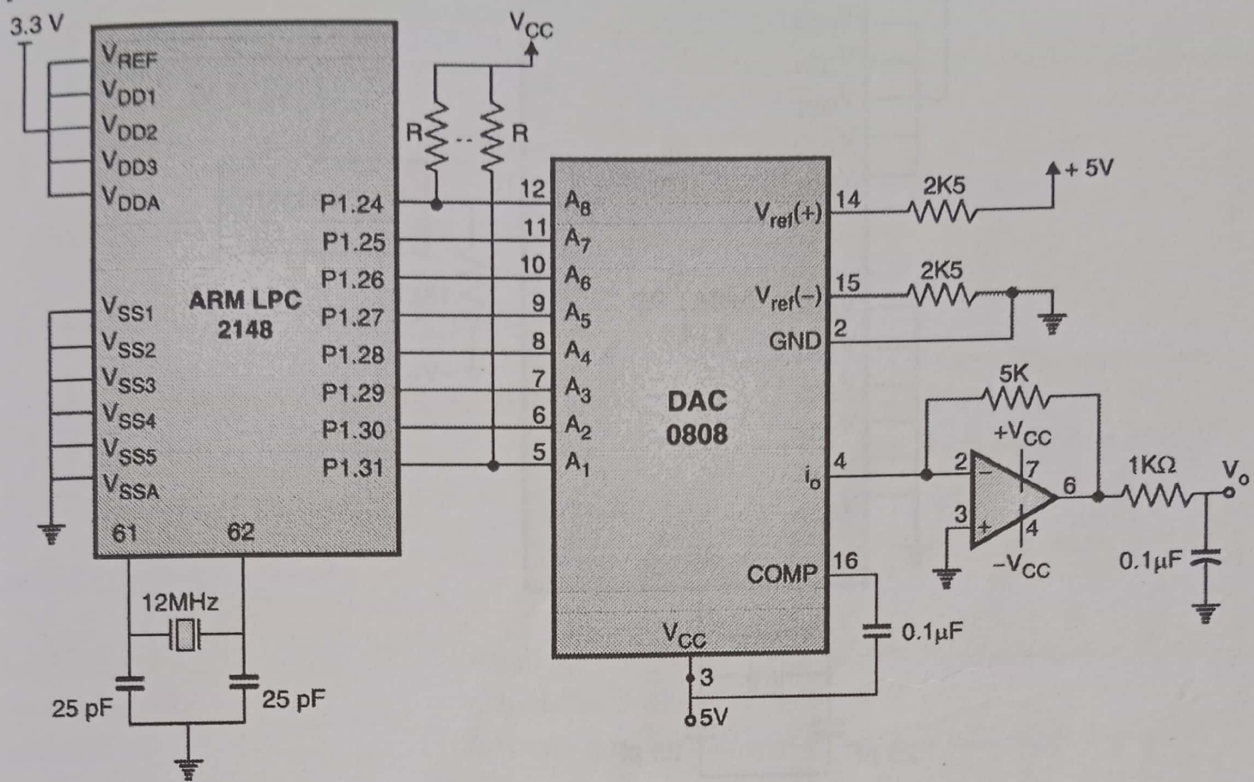


Fig. P. 3.3.3

3.3.4 Interfacing Sensors to LPC 2148

- In most of the industrial applications the parameter to be measured is temperature. Thus we need a temperature sensor that will convert the physical quantity i.e. temperature into its equivalent electrical value. They are used to measure the accurate temperature and hence control it in various industrial applications.
- The semiconductor IC LM34 is a precision temperature sensor, whose output voltage is directly proportional to the temperature in Fahrenheit. The semiconductor IC LM35 is a precision temperature sensor, whose output voltage is directly proportional to the Celsius (Centigrade) temperature.
- Another special advantage of LM35 is that it does not require any external calibration and the temperature range for which it operates is -55 to $+150^{\circ}\text{C}$.
- The LM35 is rated to operate over a -55°C to $+150^{\circ}\text{C}$ temperature range.
- As shown in the Fig. 3.3.3 the connection for LM35 is very simple. Also the output scale is linear with a change of $10\text{mV}/^{\circ}\text{C}$.

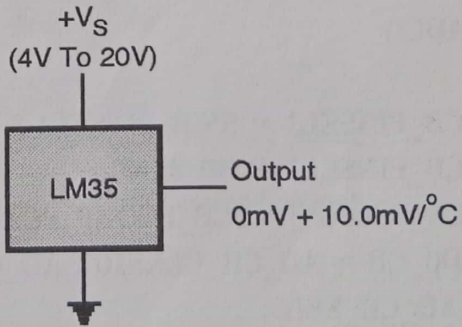


Fig. 3.3.3 : Circuit diagram for the LM35 basic temperature sensor ($+2^{\circ}\text{C}$ to $+150^{\circ}\text{C}$)

Program 3.3.4 : Write a program to read the temperature and show the interfacing of a sensor with LPC2148.

Soln. :

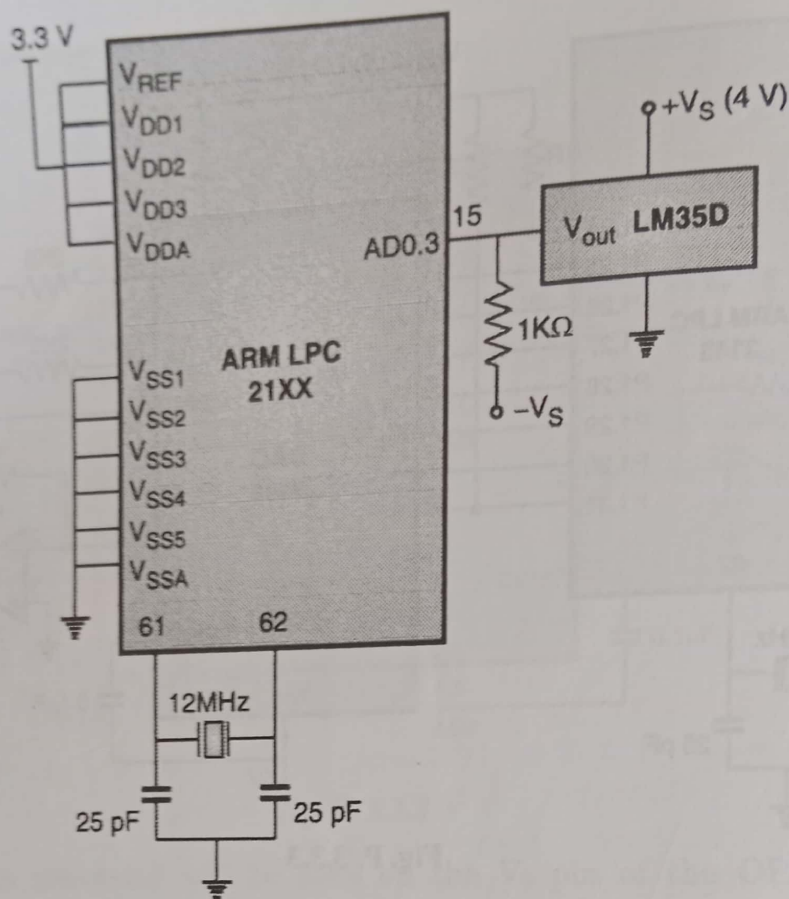


Fig. P. 3.3.4

As discussed earlier, the output of the sensor LM35 is a voltage proportional to the temperature. This analog voltage can be measured using an ADC. Thus the program for the interfacing of a temperature sensor will be same as that of the interfacing of ADC. The program is given below. The variable result holds the digital value equivalent to the temperature measured by the ADC.

```
#include "lpc214x.h"
//This is the header file that contains all the
//information about the microcontroller LPC2148
//Function to initialize ADC

void InitADC()
{
    PCB_PINSEL1 = (PCB_PINSEL1 & ~PCB_PINSEL1_P030_MASK) |
    PCB_PINSEL1_P030_AD03;
    SCB_PCONP |= SCB_PCONP_PCAD0 // Enable power supply for ADC0
    AD0_CR = AD_CR_CLKS10 | AD_CR_PDN | ((3 - 1) << AD_CR_CLKDIVSHIFT) // Initialise ADC
    | AD_CR_SEL3;
    // Enable 10-bit precision ADC, Exit power-down mode 4.0MHz & Use channel 3
}

int ReadADC()
// Function to read the digital equivalent value from ADC
```



```

{
    AD0_CR &= ~(AD_CR_START_MASK | AD_CR_SELMASK);
    //Stop all conversions and deselect all channels

    {
        AD0_CR |= (AD_CR_START_NONE | AD_CR_SEL3);
        // Select channel 3 of ADC0

        AD0_CR |= AD_CR_START_NOW;
        //Start conversion

        while (!(AD0_DR3 & AD_DR_DONE)) ;
        // Wait for the completion of the conversion

        return ((AD0_DR3 & AD_DR_RESULTMASK) >> AD_DR_RESULTSHIFT);
        // Return the digital result read from ADC
    }
}

void main()
{
    InitADC() ; // call the function to initialise ADC 0, Channel 3
    int results = 0; // Initialize the result to zero
    while (1) //Always true infinite loop to read the ADC digital data
    {
        results = ReadADC();
    }
}

```

3.4 Interfacing Motors (Stepper Motor and DC Motor)

3.4.1 Interfacing Relay

- Relay is an electrical switch that has no connection between its control input and output. Hence has a special advantage. The relay has a coil, which when excited operates from ON to OFF and vice versa when the excitement is removed.
- Fig. 3.4.1 illustrates the symbol of relay and interfacing of a relay with 8051. Simply making the pin '0' or '1' will switch ON/OFF the relay.

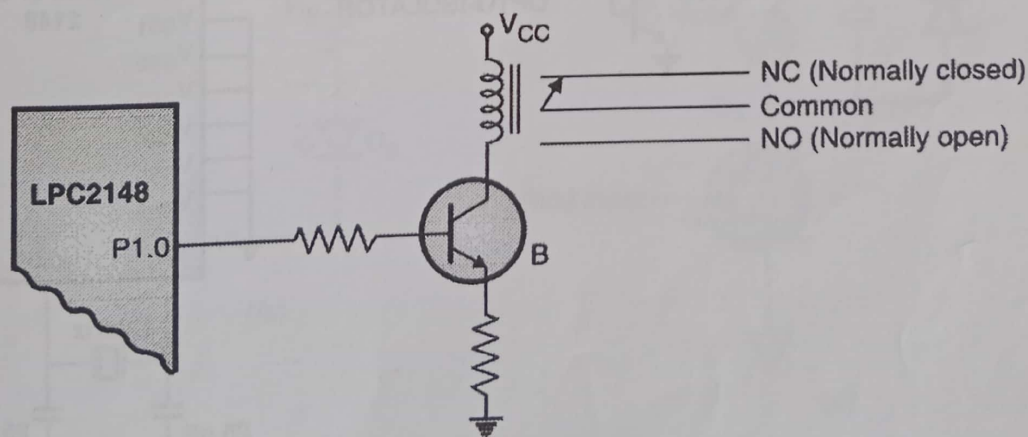


Fig. 3.4.1 : Interfacing relay with LPC2148 diagram

3.4.2 Optoisolator

- As a relay separates the input from the output electrically but coupling them magnetically; the opto-coupler also separates the input from the output electrically but coupling them optically.
- The LED at the input triggers the transistor, optically and hence no electrical path exists between the LED and transistor.
- Fig. 3.4.2 illustrates internal structure of an optoisolator and its interfacing with LPC2148

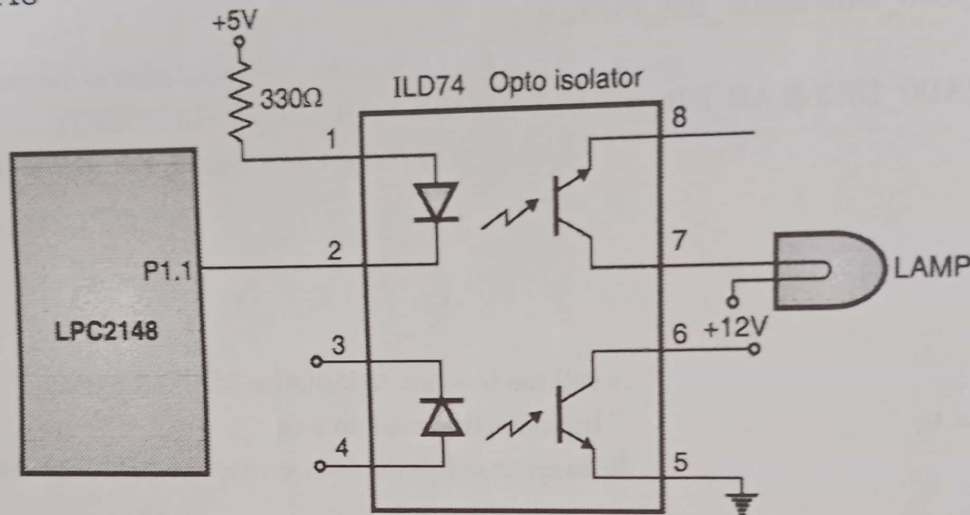


Fig. 3.4.2 : Internal structure of optoisolator and its interfacing with LPC2148

Program 3.4.1 : Write a program for switching ON and OFF a relay for a common delay and show the interfacing of the same.

Soln. :

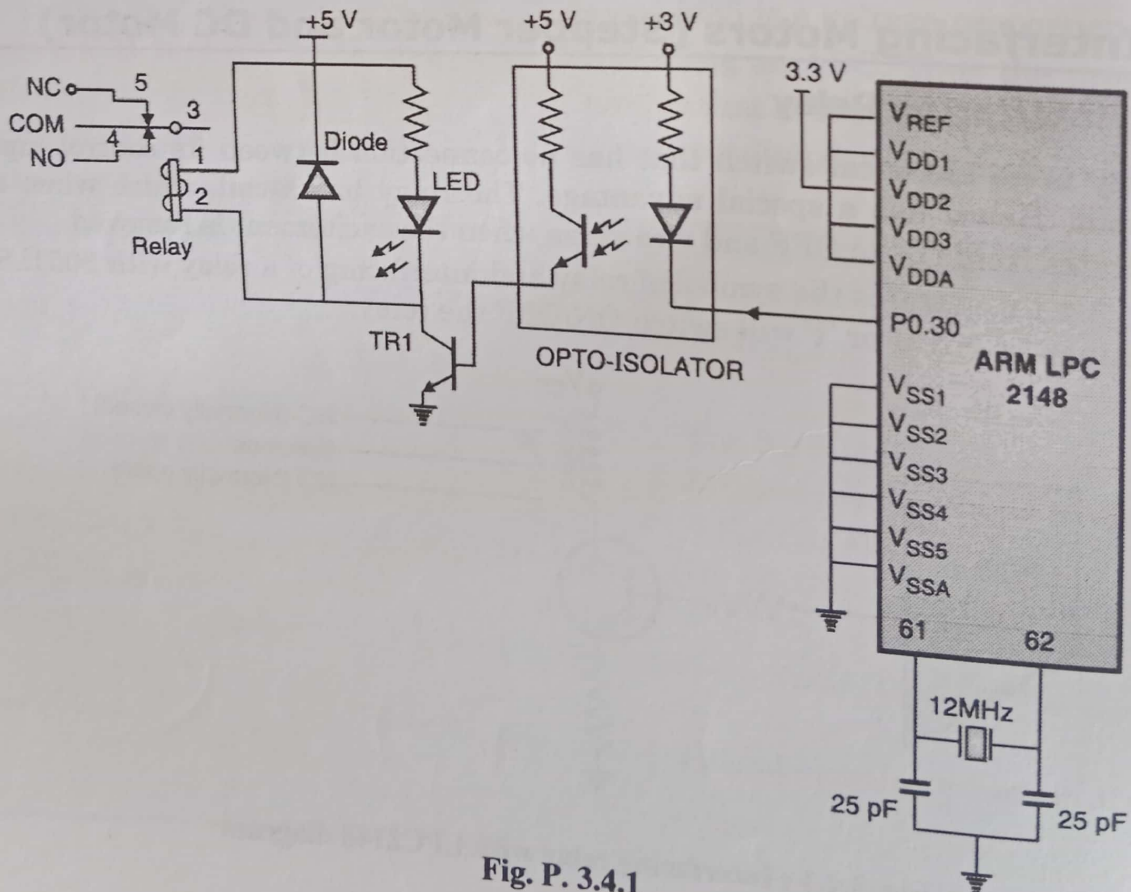


Fig. P. 3.4.1


```

#include <LPC214X.H>
#define relay 1<<30           // ASSIGN Port 0 pin 30 Pin to motor
void delay()                 // Delay function
{
    unsigned int i;
    for(i=0;i<=50000;i++);
}
void main()
{
    IODIR0=1<<30             // Configure the port 0 pin 30 as output
    while(1)
    {
        IOSET0=1<<30;        //Switch off relay
        delay();              //Call delay
        IOCLR0=1<<30;        // Switch on relay
        delay();              //Call delay
    }
}
    
```

3.4.3 Interfacing of DC Motor

- A DC motor works by converting electric power into mechanical work. This is accomplished by forcing current through a coil and producing a magnetic field that spins the motor.

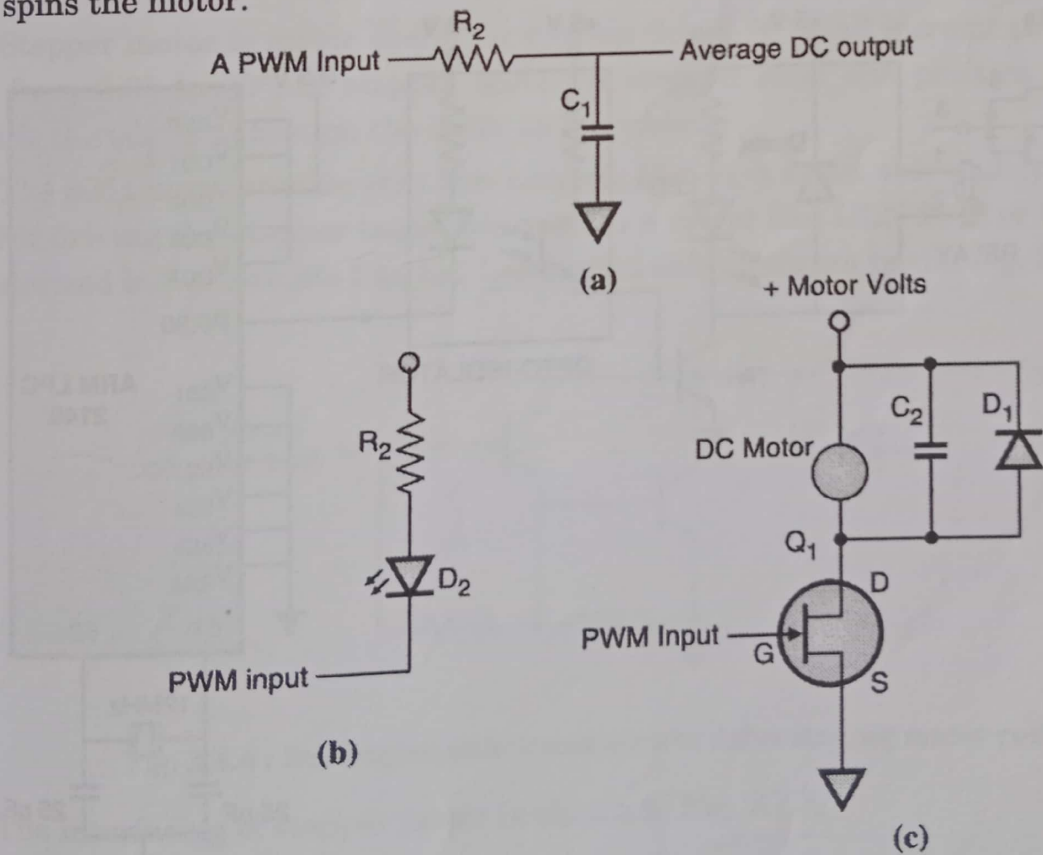


Fig. 3.4.3 : Use of PWM output

- The speed of the motor is dependent on the voltage given to it.
- Hence by varying the voltage to DC motor we can control the speed of DC motor.
- This is mainly implemented by PWM (Pulse Width Modulation)
- In PWM, a variation in the duty cycle generates variation in the average DC voltage.
- Pulse Width Modulation (PWM) is one of the methods to provide Digital to Analog Conversion, by controlling the duty cycle of the square wave provided at the output the average voltage varies and hence a DC voltage at the output of the filter available is dependent on the pulse width (counter digital value).
- A RC circuit provides filtering and gives the average DC voltage proportional to the count as shown in Fig. 3.4.3(a). The Fig. 3.4.3(b) shows how it can be used to turn on a LED and then to control its intensity by varying the voltage (inversely proportional) across it. The Fig. 3.4.3(c) shows how it can be used to turn on a DC motor and then to control its speed by varying the voltage across it.

Program 3.4.2 : Write a program for switching ON and OFF a DC motor using a relay for a common delay and show the interfacing of the same.

OR

Write a program for switching ON and OFF a relay for a common delay and show the interfacing of the same.

Soln. :

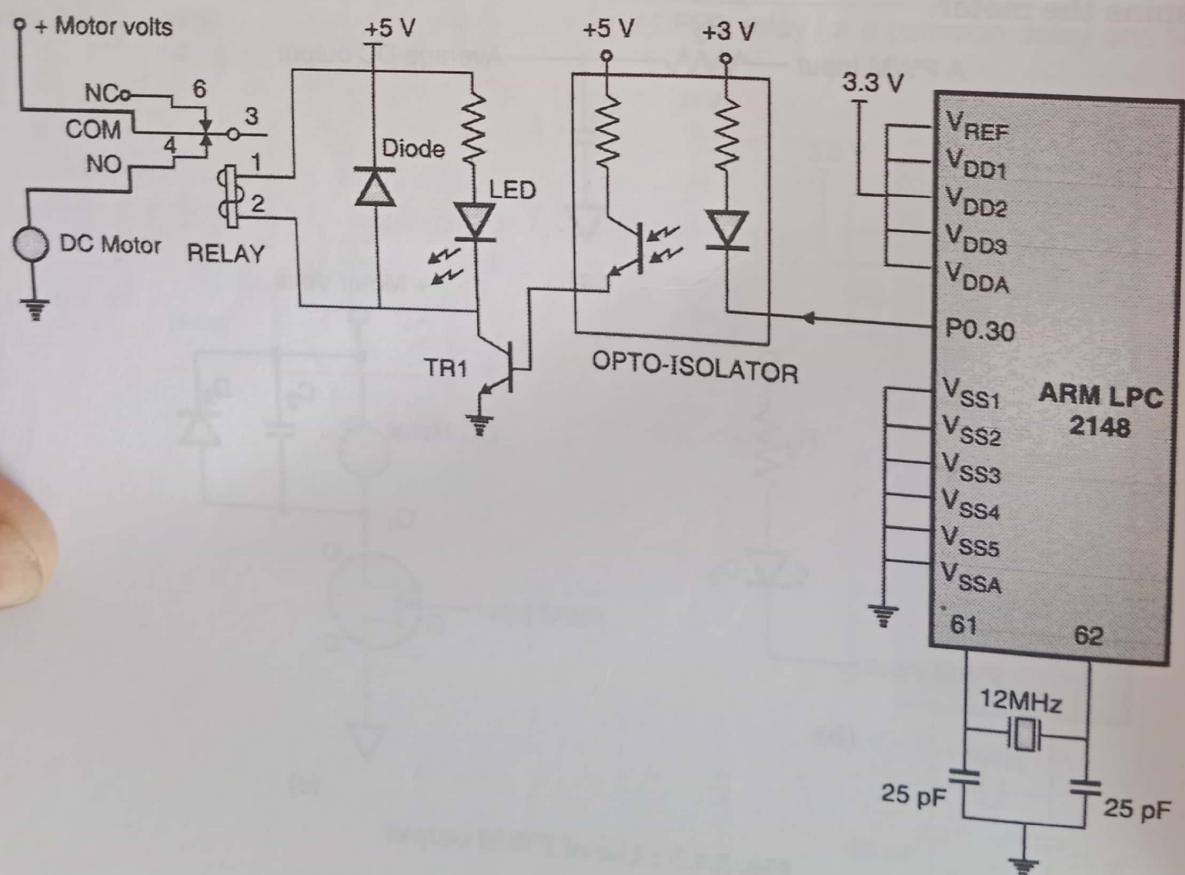


Fig. P. 3.4.2


```
#include <lpc214X.H>

void delay()                                // Delay function
{
    unsigned int i=20000;                    //by changing the delay you can control the speed of the motor
    while(i--);
}

void main()
{
    IODIR0=1<<30                            // Configure the port 0 pin 30 as output
    while(1)
    {
        IOSET0=1<<30;                        //Switch off motor
        delay();                               //Call delay function
        IOCLR0=1<<30;                        // Switch on motor
        delay();                               //Call delay function
    }
}
```

3.4.4 Interfacing Stepper Motor

Stepper motor is motor useful for moving things in small increments. The step sizes range from 0.9° to 30° . A stepper motor is stepped from one position to the next by changing the current through the fields in the motor.

The 8051 ports provide very less current. Hence, it is not able to drive stepper motor coils. For driving the stepper motor we require a driver like ULN 2003 or then we can use transistorized buffer circuits like the Darlington pair as shown in the Fig. 3.4.4.

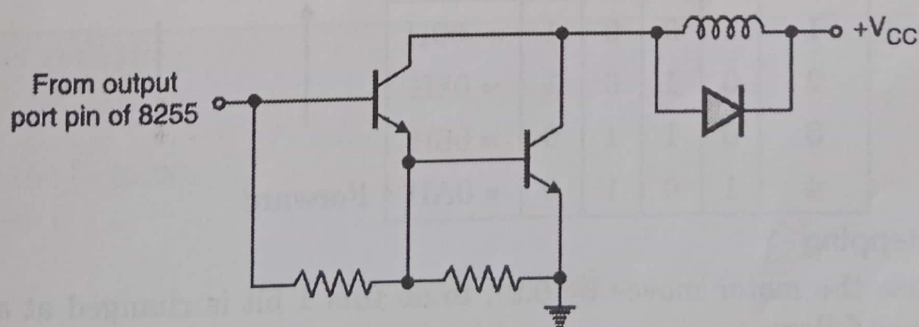


Fig. 3.4.4 : Darlington pair transistors to drive stepper motor coils

The interfacing of stepper motor is shown in Fig. 3.4.5.

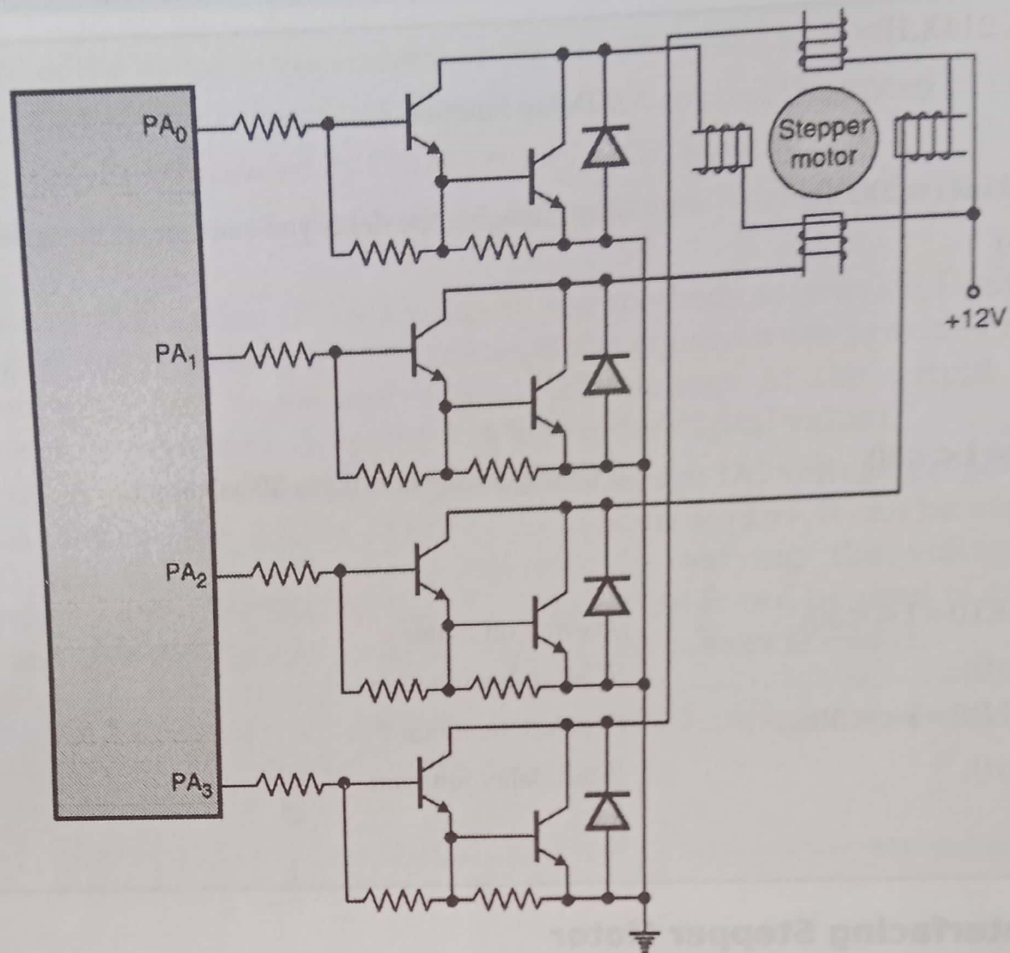


Fig. 3.4.5 : Stepper motor interface

There are two possible step modes.

Mode 1 : Full stepping

In this case the motor moves by one step (step size may vary from 0.9° to 30° depending on the motor), to do this 2 bits are changed at a time, bit patterns are as follows :

Table 3.4.1

Step	A	B	C	D	
1	1	0	0	1	= 09H
2	0	1	0	1	= 05H
3	0	1	1	0	= 06H
4	1	0	1	0	= 0AH

Reverse



Forward

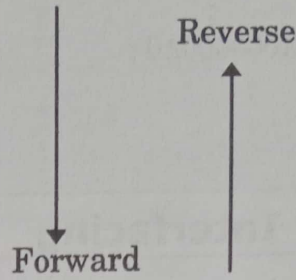
Mode 2 : Half stepping

In this case the motor moves by 0.9° , to do this 1 bit is changed at a time, the bit patterns will be as follows :

Table 3.4.2

Step	A	B	C	D	
1	1	0	0	1	= 09
2	1	0	0	0	= 08
3	1	0	1	0	= 0A

Step	A	B	C	D	
4	0	0	1	0	= 02
5	0	1	1	0	= 06
6	0	1	0	0	= 04
7	0	1	0	1	= 05
8	0	0	0	1	= 01
1	1	0	0	1	= 09



Program 3.4.3 : Write a program to drive stepper motor and show the interfacing of the same.

Soln. : ARM LPC 2148

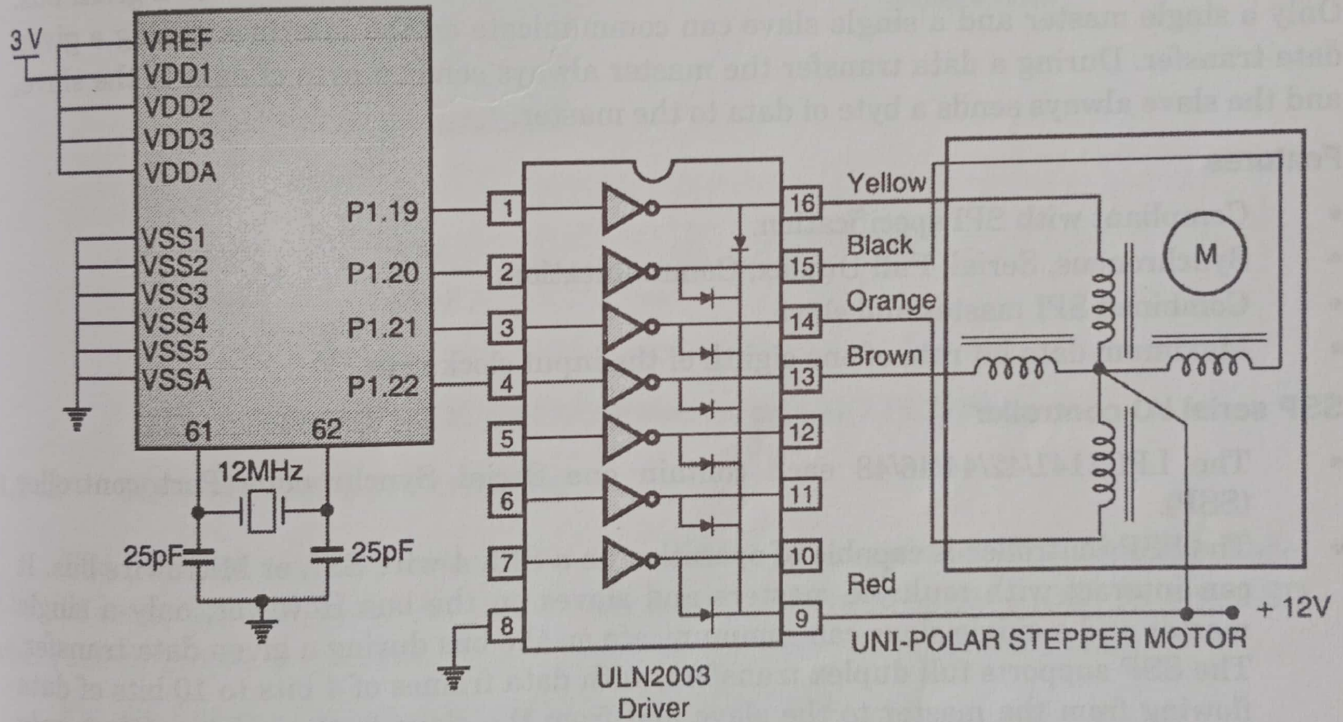


Fig. P. 3.4.3

```
#include <LPC214X.H>
void delay()
{
    unsigned int i=20000;
    while(i--);
}
// Delay function

void main() ;
// Main program starts
{
    PINSEL2 = 0X00000000;
    IO1DIR=0x000000F0 ;
    // Configure port 1 pins 19 to 22 as General Purpose I/O
    // Set Port 1 direction as output port
    while(1)
    {
        IO1PIN = 0X00000090;
        // Send the binary code for step 1
        delay();
        // Call Delay function
        IO0PIN = 0X00000050 ;
        // Send the binary code for step 2
    }
}
```




```
delay() ; // Call Delay function
IO1PIN = 0X00000060 ; // Send the binary code for step 3
delay() ; // Call Delay function
IO1PIN = 0X000000A0 ; // Send the binary code for step 4
delay() ; // Call Delay function
}
```

3.5 Memory Interfacing

SPI serial I/O controller

The LPC2141/42/44/46/48 each contain one SPI controller. The SPI is a full duplex serial interface, designed to handle multiple masters and slaves connected to a given bus. Only a single master and a single slave can communicate on the interface during a given data transfer. During a data transfer the master always sends a byte of data to the slave, and the slave always sends a byte of data to the master.

Features

- Compliant with SPI specification.
- Synchronous, Serial, Full Duplex, Communication.
- Combined SPI master and slave.
- Maximum data bit rate of one eighth of the input clock rate.

SSP serial I/O controller

- The LPC2141/42/44/46/48 each contain one Serial Synchronous Port controller (SSP).
- The SSP controller is capable of operation on a SPI, 4-wire SSI, or Microwire bus. It can interact with multiple masters and slaves on the bus. However, only a single master and a single slave can communicate on the bus during a given data transfer. The SSP supports full duplex transfers, with data frames of 4 bits to 10 bits of data flowing from the master to the slave and from the slave to the master. Often only one of these data flows carries meaningful data.

Features

- Compatible with Motorola's SPI, TI's 4-wire SSI and National Semiconductor's Microwire buses.
- Synchronous serial communication.
- Master or slave operation.
- 8-frame FIFOs for both transmit and receive.
- Four bits to 16 bits per frame.

3.5.1 Interfacing I2C EEPROM with LPC2148

PU - May 2015

The I2C (Inter-IC) bus is a bi-directional serial communication protocol used for communication between the integrated circuits (ICs). It is developed by Phillips and is a synchronous protocol for serial communication. LPC2148 has a I2C port, hence it can be easily interfaced with the I2C EEPROM. Fig. 3.5.1 shows the interfacing of LPC2148 with I2C EEPROM.

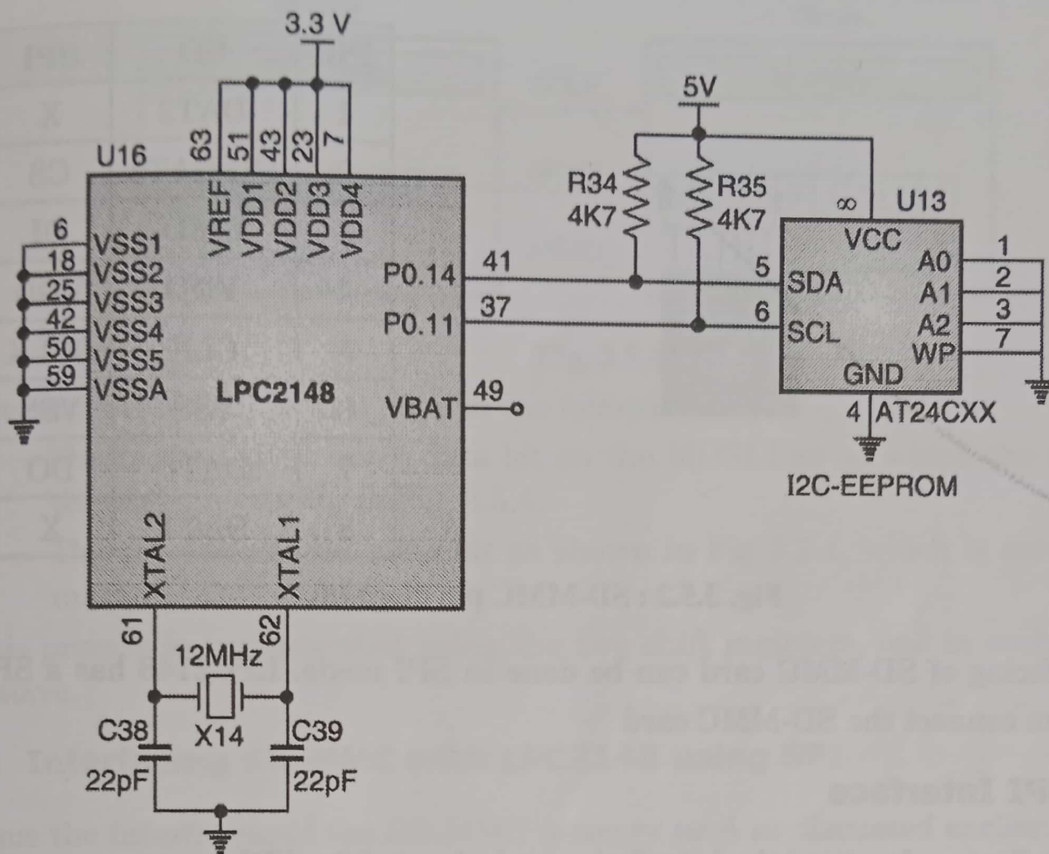
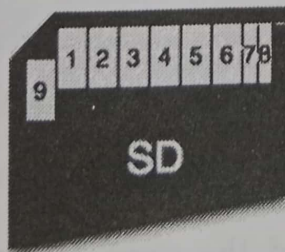


Fig. 3.5.1 : I2C EEPROM interfacing with LPC2148

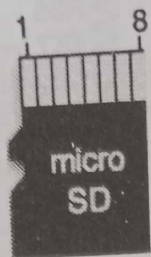
3.5.2 SD-MMC Card Interfacing

The SD card can communicate in three different transfer modes: 1-bit SD mode, 4-bit SD mode and SPI mode. All cards must support all three modes except for micro SD where the SPI mode is optional. We will interface the micro SD in the SPI mode.

Fig. 3.5.2 shows the pin diagram of the SD-MMC card.



Pin	SD	SPI
1	CD/DAT3	CS
2	CMD	DI
3	VSS	VSS1
4	VDD	VDD
5	CLK	SCLK
6	VSS2	VSS2
7	DAT0	D0
8	DAT1	X
9	DAT2	X



Pin	SD	SPI
1	DAT2	X
2	CD/DAT3	CS
3	CMD	DI
4	VDD	VDD
5	CLK	SCLK
6	VSS	VSS
7	DAT0	DO
8	DAT1	X

Fig. 3.5.2 : SD-MMC pin diagram

Interfacing of SD-MMC card can be done in SPI mode. LPC2148 has a SPI port to which we can connect the SD-MMC card

3.5.2.1 SPI Interface

Let us first understand the interfacing technique of the SPI.

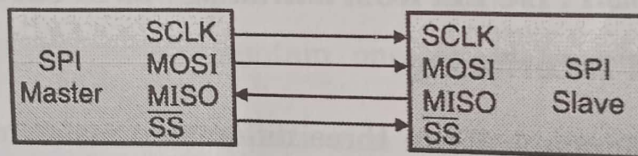


Fig. 3.5.3

The Serial Peripheral Interface (SPI) bus is a synchronous serial communication protocol that supports full duplex (simultaneous bi-directional) communication. The SPI communication works in a special manner wherein, one device works as a master while the other works as slave. There can also be multiple slave devices.

The SPI bus specifies four logic signals :

- SCLK: Serial Clock (output from master);
- MOSI; SIMO: Master Output, Slave Input (output from master);
- MISO; SOMI: Master Input, Slave Output (output from slave);
- SS: Slave Select (active low, output from master).

Data transmission between the master and the slave is done as shown in the Fig. 3.5.4.

In this case the first task is the configuration of clock which is done by the master device. The slave device is then selected by the master. The frequency commonly used is in the range of 1 to 70 MHz.

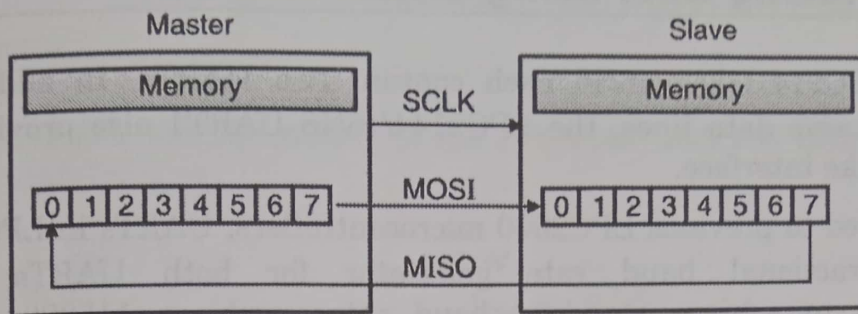


Fig. 3.5.4

The communication is then done in the following steps:

1. The master gives out a data bit on the MOSI line on which the slaves reads this data, as shown in Fig. 3.5.4.
2. The slave sends the data bit as shown in Fig 3.5.4, which is received by the master.

This process is implemented using the two shift registers, one in each the master and the slave.

3.5.2.2 Interfacing SD-MMC with LPC2148 using SPI

Thus the interfacing of the SD-MMC memory card as discussed earlier can be done using the SPI protocol with LPC2148. Fig. 3.5.5 shows the interfacing of the SD-MMC card with LPC2148.

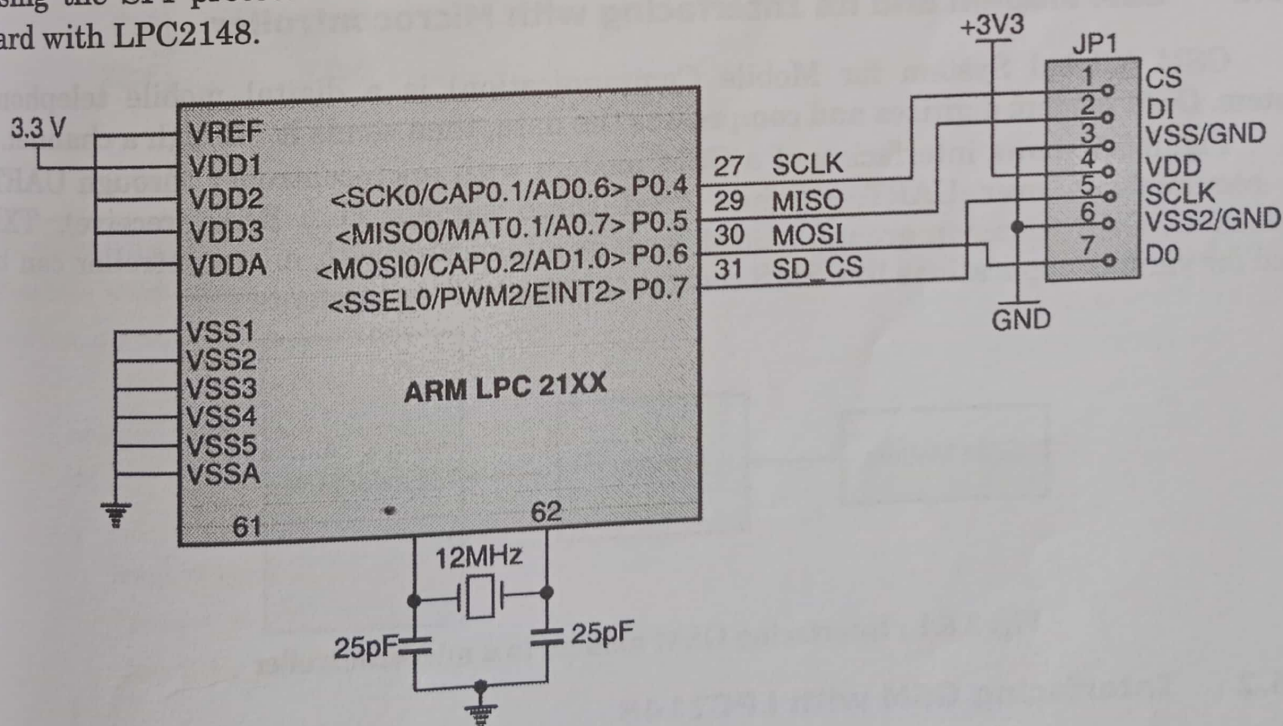


Fig. 3.5.5 : SD-MMC card interfacing with LPC2148

The SD-MMC card can be connected at the jumper shown at the right end in the Fig. 3.5.5. LPC2148 has a SPI port whose pins are available on port 0 pins. Hence the SD card is connected on this port pins of LPC2148. These pins are port 0 pins 4, 5, 6 and 7 working as SCLK, MISO, MOSI and SD_CS respectively.

3.6 Interfacing GSM using UART

† The LPC2141/42/44/46/40 each contain two UARTs. In addition to standard transmit and receive data lines, the LPC2144/46/48 UART1 also provides a full modem control handshake interface.

† Compared to previous LPC2000 microcontrollers, UARTs in LPC2141/42/44/46/48 introduce a fractional baud rate generator for both UARTs, enabling these microcontrollers to achieve standard baud rates such as 115200 with any crystal frequency above 2 MHz. In addition, auto-CTS/RTS flow-control functions are fully implemented in hardware (UART1 in LPC2144/46/48 only).

Features

- 16 B Receive and Transmit FIFOs
- Register locations conform to 16C550 industry standard.
- Receiver FIFO trigger points at 1 B, 4 B, 8 B, and 14 B
- Built-in fractional baud rate generator covering wide range of baud rates without a need for external crystals of particular values.
- Transmission FIFO control enables implementation of software (XON/XOFF) flow control on both UARTs.
- LPC2144/46/48 UART1 equipped with standard modem interface signals. This module also provides full support for hardware flow control (auto-CTS/RTS).

3.6.1 GSM Modem and its Interfacing with Microcontroller

GSM (Global System for Mobile Communication) is a digital mobile telephony system. GSM modem digitizes and compresses the data, then sends it through a channel.

Fig. 3.6.1 shows interfacing of a GSM modem with microcontroller through UART. To communicate over UART, we just need three signals viz., RXD (receive), TXD (transmit), GND (common ground). GSM modem interfacing with microcontroller can be used for various applications like SMS control of industrial equipments.

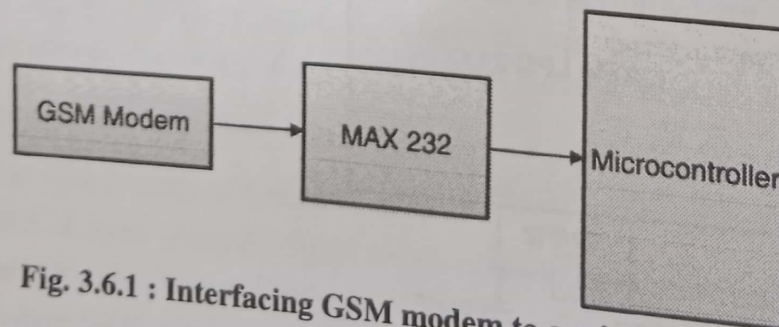


Fig. 3.6.1 : Interfacing GSM modem to a microcontroller

3.6.2 Interfacing GSM with LPC2148

- In LPC2148 microcontroller we have two serial interfaces that i.e. the UART0 and the UART1. We can use either of them, but in the following example we will be using UART0.

† Courtesy NXP/Philips

Program 3.6.1 : Write a program to send a SMS using a GSM modem and show the interface for the same.

Soln. :

- Fig. P. 3.6.1 shows the interfacing of the LPC2148 with the GSM modem through MAX232 a UART driver.

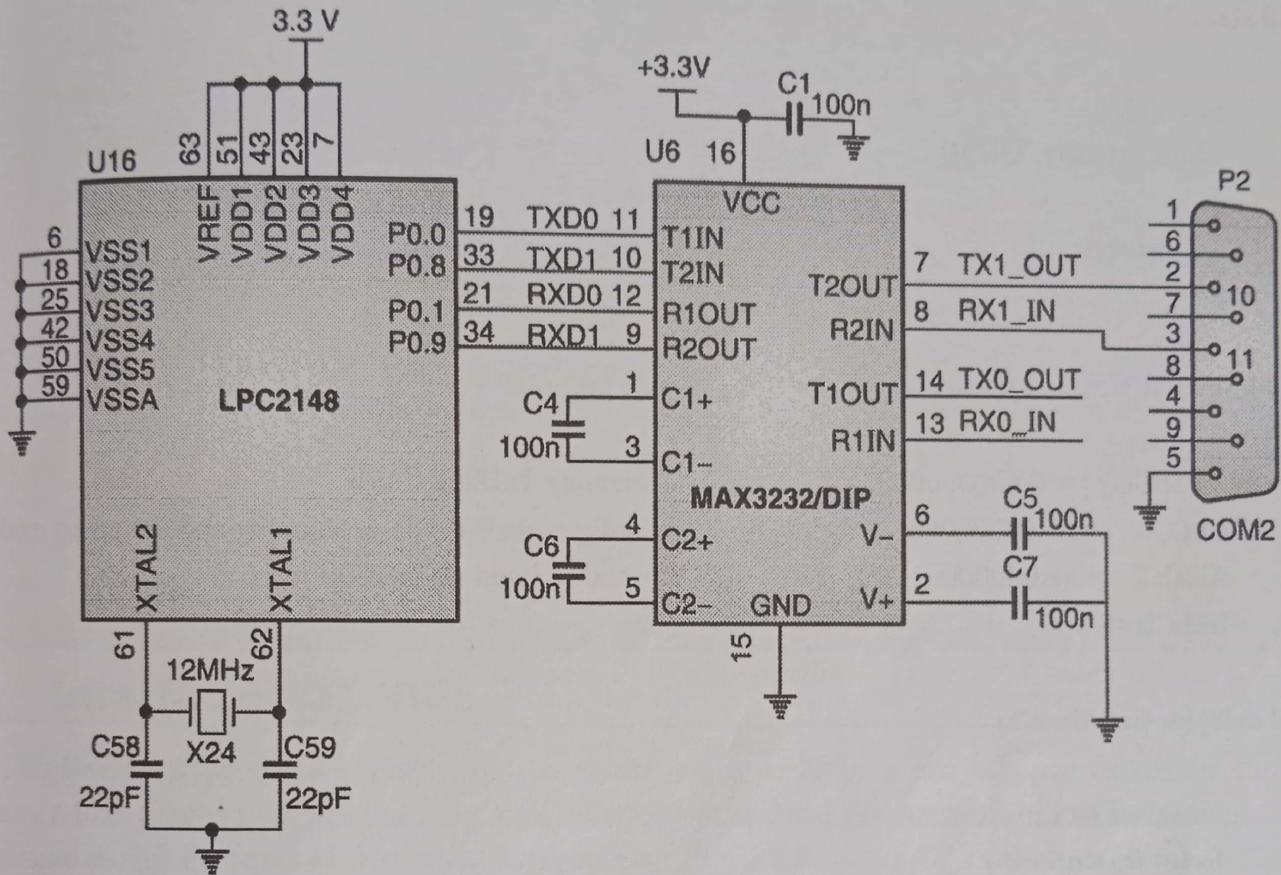


Fig. P. 3.6.1

```
#include <lpc21xx.H>
#include <stdio.h>
void input(unsigned char *array)
{
    unsigned char temp=0, i=0;
    do
    {
        temp = getchar();
        *array = temp;
        array++;
    }
    while((temp != '\r') && (temp != '\n'));
    *array = '\0';
}

char getchar () /* Read a character from the serial Port
{
    while (!(U0LSR & 0x01))
```



```

    {
    }
    return (U0RBR);
}

void status()
{
    input(y);
    while(!(strstr(y,"OK")))
    {
        input(y);
    }
}

void InitSerial()
{
    PINSEL0 |= 0X00000005;           //Enabling TxD0 & RxD0
    U0LCR = 0x00000083;             //Configure in 8-bit data, no parity and 1-stop bit mode
    U0DLL = 0x00000061;             //Initialize baud rate=9600
    U0LCR = 3;
}

void delay(unsigned int n)
{
    unsigned int i,j;
    for(i=0;i<n;i++)
    for(j=0;j<10000;j++);
}

void main()
{
    unsigned int cnt=0x80,m;
    char xx;
    InitSerial();
    delay(50);
    while(1)
    {
        printf("AT\r");
        status();
        printf("AT+IPR=9600\r");
        status();
        printf("AT+CMGR=2\r");
        status();
        delay(250);
        // Send GSM AT command for initializing GSM modem
        // Check is sent properly.
        // Send GSM AT command to initialize the baud rate
        // Send command to read the message detail
        // at Index 1 with phone number, data and time
        // Call delay function
    }
}

```



```
printf("ATD9790550124;\r"); //Send command for call dialing
delay(250);
status();
delay(3500);
printf("ATH\r"); // Send command for call disconnecting
delay(250);
status();
delay(1000);
printf("ATDL\r"); // Send AT command for redialing
delay(250);
status();
delay(1000);
printf("ATH\r"); // Send AT command for answering the call
delay(250);
status();
delay(1000);
}
```

3.7 Interfacing ZIGBEE

ZigBee is a protocol for wireless communication. It is a set of specifications that consumes low-power and is based on an IEEE 802 standard for personal area networks. It is a simple serial method of transmit and receive.

Interfacing ZigBee with a microcontroller is shown in Fig. 3.7.1. It is interfaced using a simple serial port and the MAX232 for shifting the voltage levels. These operate at a frequency of 2.4 GHz. We have seen in the previous section how RS232 works, as well as how the SPI protocol works.

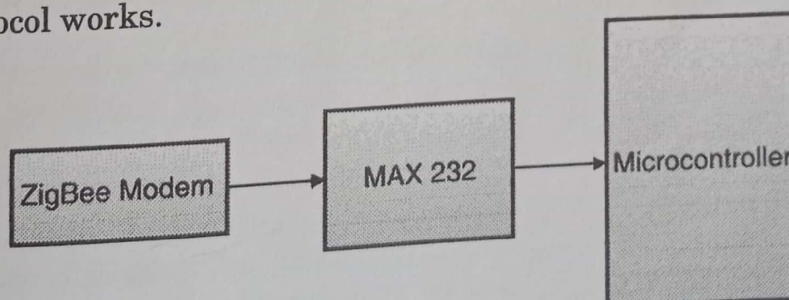


Fig. 3.7.1

Program 3.7.1 : Write a program to receive the data through a ZigBee module and transmit the same using a serial port; also show the interfacing of the same.

Soln. :

The circuit diagram shown in Fig. P. 3.7.1 is used for the interfacing of the ZigBee module and it is same as that in the interfacing of the GSM modem. This is because both of them use the SPI protocol for communication in the interfacing seen by us.

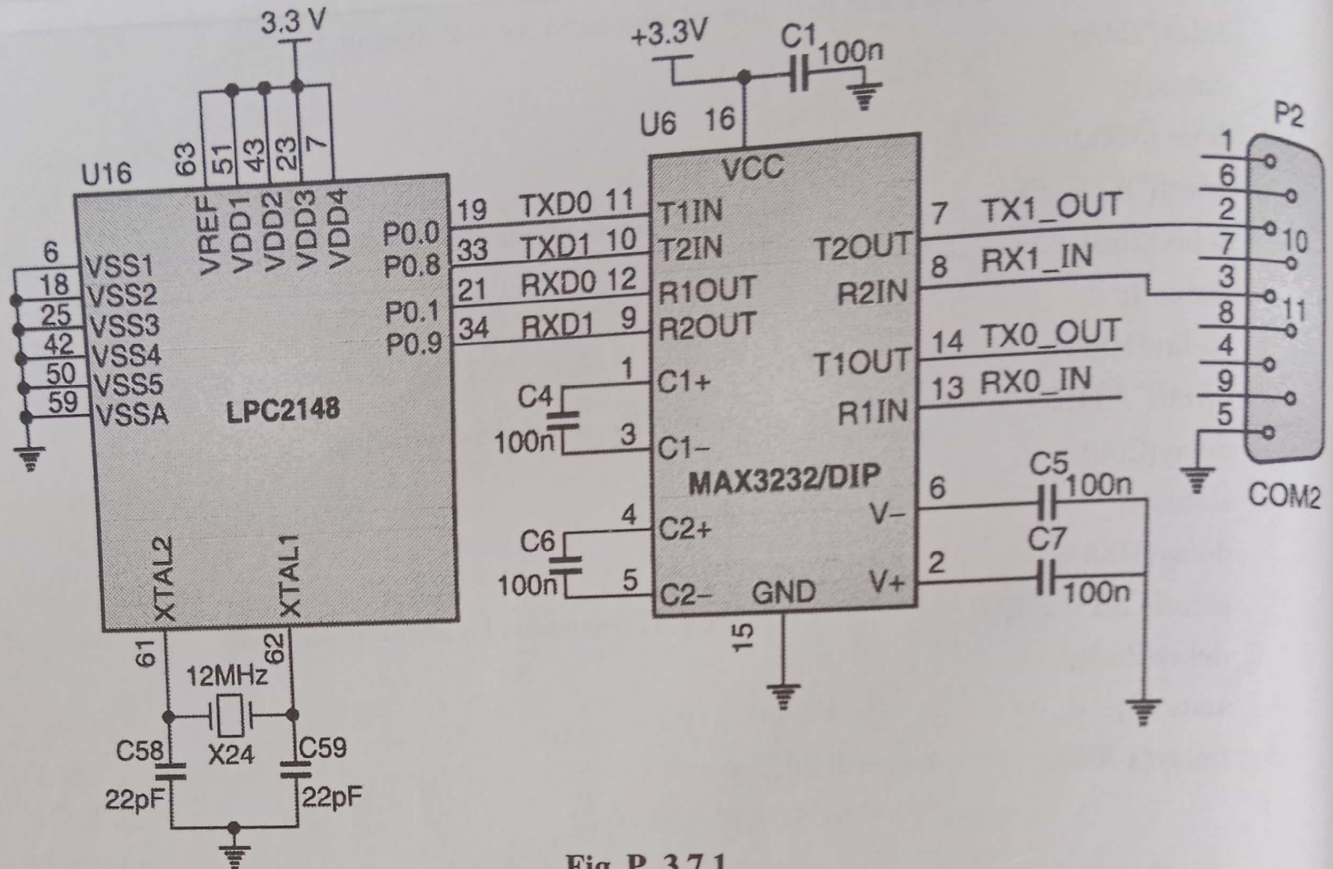


Fig. P. 3.7.1

```
#include <lpc21xx.H>
unsigned char test;
void InitSerial()
{
    PINSEL0 |= 0X00000005; //Enabling TxD0 & RxD0
    U0LCR = 0x00000083; //Configure in 8-bit data, no parity and 1-stop bit mode
    U0DLL = 0x00000061; //Initialize baud rate=9600
    U0LCR = 3;
}
char send (char ch)
{
    if (ch == '\n')
    {
        while (!(U0LSR & 0x20))
        {
        }
        U0THR = 0x0D;
    }
    while (!(U0LSR & 0x20))
    {
    }
    return (U0THR = ch);
}
char input ()
```



```

{
    while (!(UOLSR & 0x01))
    {
    }
    return (UORBR);
}

int main()
{
    char *p = "Enter characters to be sent:\n\r";
    VPBDIV = 0x02; // Initialize for division of PCLK by two
    InitSerial();
    while(1)
    {
        while (*p)
        {
            send(*p);
            p++;
        }
        send(input()); // Echo terminal
    }
}

```

3.8 Interfacing Finger Print (Biometric)

For interfacing a biometric system like the fingerprint recognition, we need to have a module that can sense the finger prints and perform various operations on the same. We will be using the SM630 biometric module as it is very easy to interface the same.

3.8.1 SM630 Biometric Module

SM630 is a optical fingerprint verification module released by Maxis Biometrics Co., Ltd. It consists of an optical sensor, a DSP processor and Flash memory. It also contains functions such as login through fingerprint, deletion of fingerprint record, verification of fingerprint, uploading a new fingerprint, downloading of a fingerprint record, etc.

Fig. 3.8.1 shows an image of the SM630 biometric sensor.

The interfacing of this module can be done using a simple serial port connection. The module has 4 connections i.e. 2 pins for power supply and 2 pins for full duplex serial communication. The Table 3.8.1 gives the description of the pins.



Fig. 3.8.1 : SM630 module

Table 3.8.1

Sr. No.	PIN definition	Remarks
1.	Power supply +	Power supply +
2.	Module Tx	Open-drain output, need to use pull-up resistance in application (Typical value 10K Ω)
3.	Module Rx	Wide voltage input, 7V affordable
4.	Power supply	Power supply

3.8.2 Interfacing the SM630 Biometric Module with LPC2148

- As seen earlier, to interface the biometric module with the LPC2148 microcontroller, we need to simply use the serial port of the microcontroller.
- There are various functions that the module can perform. To perform these operations there are some codes or commands to be given to it. The command list for the module is given in the Table 3.8.2. The name of the command in the table is self explanatory so as to understand what is done by that function.

Table 3.8.2 : Command list for SM630 module

Sr. No.	Name of command	Command code
1.	Add fingerprint	0x40
2.	Delete fingerprint	0x42
3.	Search fingerprint	0x44
4.	Empty fingerprint database	0x46
5.	Search information in fingerprint database	0x4B
6.	Download fingerprint template	0x50
7.	Upload fingerprint template	0x52
8.	Upload fingerprint template	0x60
9.	Read user flash	0x62
10.	Write user flash	0x64
11.	Read product logo	0x80

For each command given to the module, it responds with a response code. The Table 3.8.3 gives the list of response code given by the SM630 module for various commands.

Table 3.8.3 : Response code

Sr. No.	Name of command	Response code
1.	Receive correct	0x01
2.	Receive error	0x02
3.	Operation successful	0x31
4.	Finger detected	0x32
5.	Time out	0x33
6.	Fingerprint process failure	0x34

Sr. No.	Name of command	Response code
7.	Parameter error	0x35
8.	Fingerprint matching with this ID found	0x37
9.	No matching fingerprint with this ID	0x38
10.	Fingerprint found	0x39
11.	Fingerprint unfound	0x3A

The algorithm to add a new finger print is as given below:

1. The HOST has to send command for adding fingerprint at position 0. The command has to be given in the following sequence:
0x4D, 0x58, 0x10, 0x03, 0x40, 0x00, 0x00, 0xF8
2. The module will respond for status indicating correct reception in the following sequence:
0x4D, 0x58, 0x30, 0x01, 0x01, 0xD7
3. The fingerprints are to be confirmed, so the finger is to be pressed in the given slot and to this the module will respond with status in following sequence:
0x4D, 0x58, 0x30, 0x02, 0x40, 0x31, 0x48
4. The procedure in third step above is to be repeated for second confirmation of the finger print, to which the module will respond in following sequence:
0x4D, 0x58, 0x30, 0x02, 0x40, 0x31, 0x48

Program 3.8.1 : Write a program to add a finger print in the biometric module and also show the interfacing of the same.

Soln. :

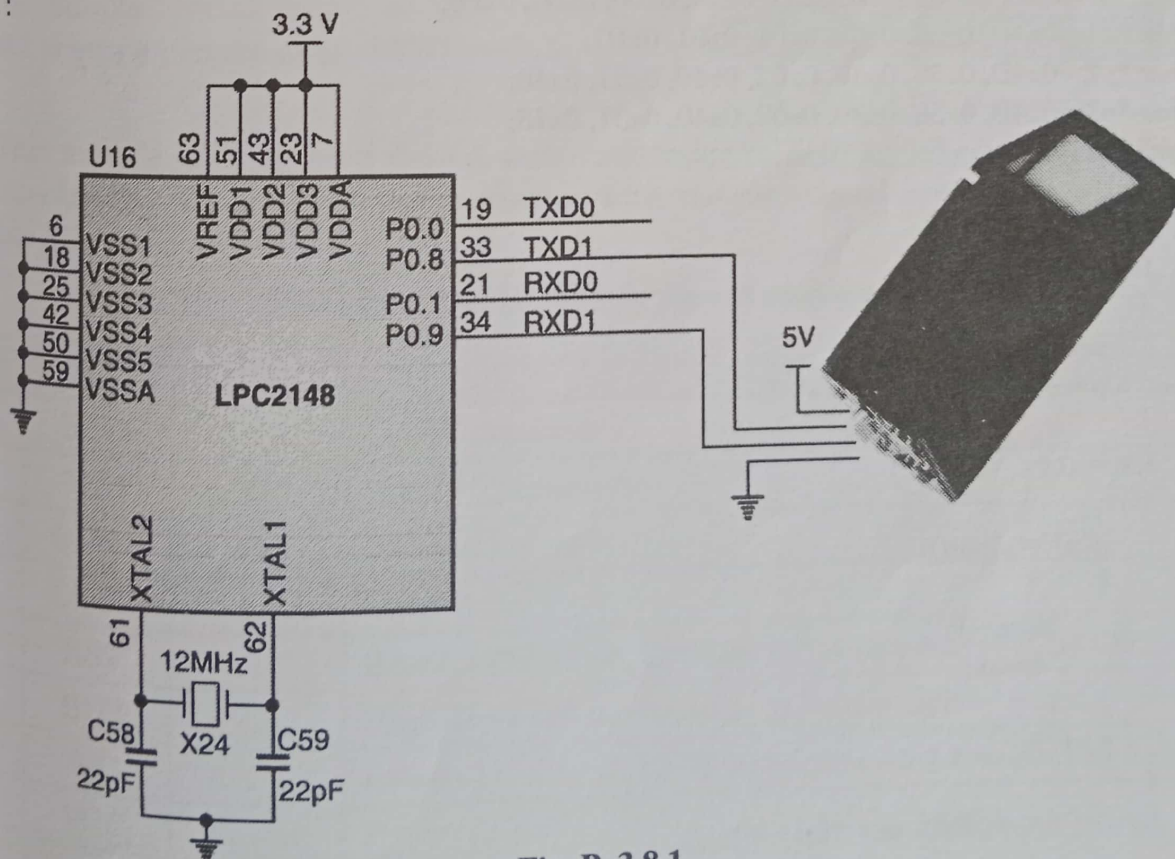


Fig. P. 3.8.1



```
#include <lpc21xx.H>
```

```
unsigned char test;
```

```
void InitSerial()
```

```
{
```

```
    PINSEL0 |= 0X00000005;
```

```
    U0LCR = 0x00000083;
```

```
    U0DLL = 0x00000061;
```

```
    U0LCR = 3;
```

```
}
```

```
void send (int ch)
```

```
{
```

```
    while (!(U0LSR & 0x20));
```

```
    U0THR = ch;
```

```
}
```

```
char input ()
```

```
{
```

```
    while (!(U0LSR & 0x01))
```

```
    {
```

```
    }
```

```
    return (U0RBR);
```

```
}
```

```
int main()
```

```
{
```

```
    VPBDIV = 0x02;
```

```
    int i,flag=0;
```

```
    char *p=0x4D, 0x58, 0x10, 0x03, 0x40, 0x00, 0x00, 0xF8;
```

```
    char *p1=0x4D, 0x58, 0x30, 0x01, 0x01, 0xD7;
```

```
    char *p2=0x4D, 0x58, 0x30, 0x02, 0x40, 0x31, 0x48;
```

```
    char *p3=0x4D, 0x58, 0x30, 0x02, 0x40, 0x31, 0x48;
```

```
    InitSerial();
```

```
    while(1)
```

```
    {
```

```
        for(i=0;i<=7;i++)
```

```
        {
```

```
            send(*p);
```

```
            p++;
```

```
        }
```

```
        for(i=0;i<=5;i++)
```

```
        {
```

```
            if(p1!=input())
```

```
            {
```

```
                flag=1;
```

```
                break;
```

```
            }
```

```
            p1++;
```

```
        }
```

```
        for(i=0;i<=6&&flag==0;i++)
```

```
        {
```

```
//Enabling TxDO & RxDO
```

```
//Configure in 8-bit data, no parity and 1-stop bit mode
```

```
//Initialize baud rate=9600
```

```
//wait for previous transmission to complete
```

```
//Give new data to be transmitted
```

```
//wait for data to be received
```

```
//return the received data
```

```
// Initialize for division of PCLK by two
```



```

    send(*p2);
    p2++;
}
for(i=0;i<=6&&flag==0;i++)
{
    if(p3!=input())
    {
        flag=1;
        break;
    }
    p3++;
}
}

```

In the above program the variable flag will indicate whether the entry is properly added or not.

3.9 Interfacing RFID

There are various RFID modules available and are useful in various applications. One of the widely used RFID module is the YHY502CTG, 13.56 MHz RFID module. The interfacing of the RFID module is also done using a simple serial port or the UART. Fig. 3.9.1 shows a picture of an RFID module.

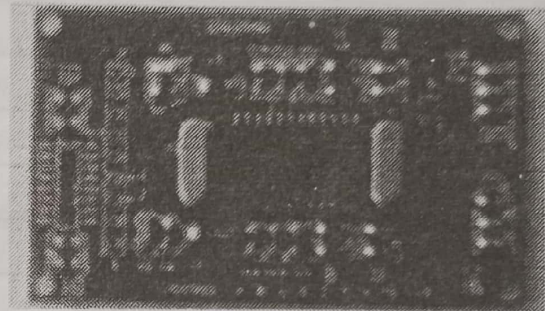


Fig. 3.9.1 : RFID module

This module is integrated with a microcontroller to help serial communication and other interfacing. It has a nine pin connector with the pin connection details as shown in the Table 3.9.1.

Table 3.9.1 : RFID module pin connections

Pin	Symbol	IO Type	Description
1.	RXD	I	UART receiver
2.	TXD	O	UART Transmitter
3.	OUT1	O	Output 1
4.	OUT2	O	Output 2
5.	RST	I	Reset, active-low, floating for power-on reset by default
6.	BUZ	O	High level drive, connect to buzzer drive circuit
7.	SIG	O	Interrupt output, LOW level indicates card in the field
8.	VCC	Power	Power positive
9.	GND	GND	Power negative



There are various commands required to communicate with this module. A list of these commands is given in the Table 3.9.2.

Table 3.9.2 : Command set of YHY505CTG

Code	Command	Description
0x01	Module Type	Read Module Type
0x02	Module serial Number	Read module serial number
0x03	Power Down	Set Module power down
0x10	Firmware version	Read module firmware version
0x11	Antenna control	Set module antenna on or off
0x12	Card IDLE	Set card IDLE
0x13	Seek	Set Auto-search card
0x14	Beep	Set Buzzer ON/OFF
0x15	Beep interval	Set buzzer beep interval time
0x16	Output1	Set output 1
0x17	Output2	Set output 2
0x19	Card type	Read Card type
0x20	Card serial number	Read card serial number
0x21	Block read	Read card Block data, 16 bytes
0x22	Block write	Write card Block data, 16 bytes
0x23	Initialize epurse	Initialize one block into epurse value
0x24	Value read	Read ePurse value, 4 bytes
0x25	Increment	Increase ePurse value, 4 bytes
0x26	Decrement	Decrease Purse value, 4 bytes
0x32	Read E ²	Read Module's EEPROM
0x33	Write E ²	Write Module's EEPROM

It uses UART protocol with a baud rate of a maximum of 19200, 8 bits data, one stop bit and no parity bit. Each data given to the module should be in the packet format shown in the Fig. 3.9.2.

Header	Length	Command	Data	CSUM
2 Byte	1 Byte	1 Byte	N Bytes	1 Byte

Fig. 3.9.2 : Format of the packet to be given to the RFID module

The header field is made up of 2 bytes and should always be 0xAA and 0xBB. The length field indicates the size or the number of bytes in the data field. Then the command is given which consists of the command given in the table above. The data then follows, which will be required for reading or writing the data. The last field is of one byte and is

the check sum byte. The check sum byte is used for checking if data is corrupted. The checksum byte is calculated by EX-ORing the various fields i.e. using the following formula.

$$CSUM = Length \oplus Command \oplus Data[0] \oplus Data[1] \dots \oplus Data[n - 1]$$

An important point to be noted is that, if any byte is 0xAA, then it is to be suffixed with a 0x00.

After every command information sent to the RFID module as shown in the above format, the module returns a packet in the format shown in the Fig. 3.9.3.

Header	Length	Status	Response	CSUM
2 Byte	1 Byte	1 Byte	N Bytes	1 Byte

Fig. 3.9.3 : Format of the packet received from the RFID module

The header and the length field for the received packet is also same as the transmitted packet. The status is returned is same as the command, if the command has been executed properly. If the operation is not performed properly then the ones complement of the command is returned as the status. The response field returns the data if any required according to the command, else it is empty. The checksum is again calculated in a similar manner as shown by the formula below.

$$CSUM = Length \oplus Command \oplus Response[0] \oplus Response[1] \oplus \dots \oplus Response [n - 1]$$

Let us see a program example to read the module type of the RFID.

Program 3.9.1 : Write a program to read the RFID module type; also show the interfacing for the same.

Soln. :

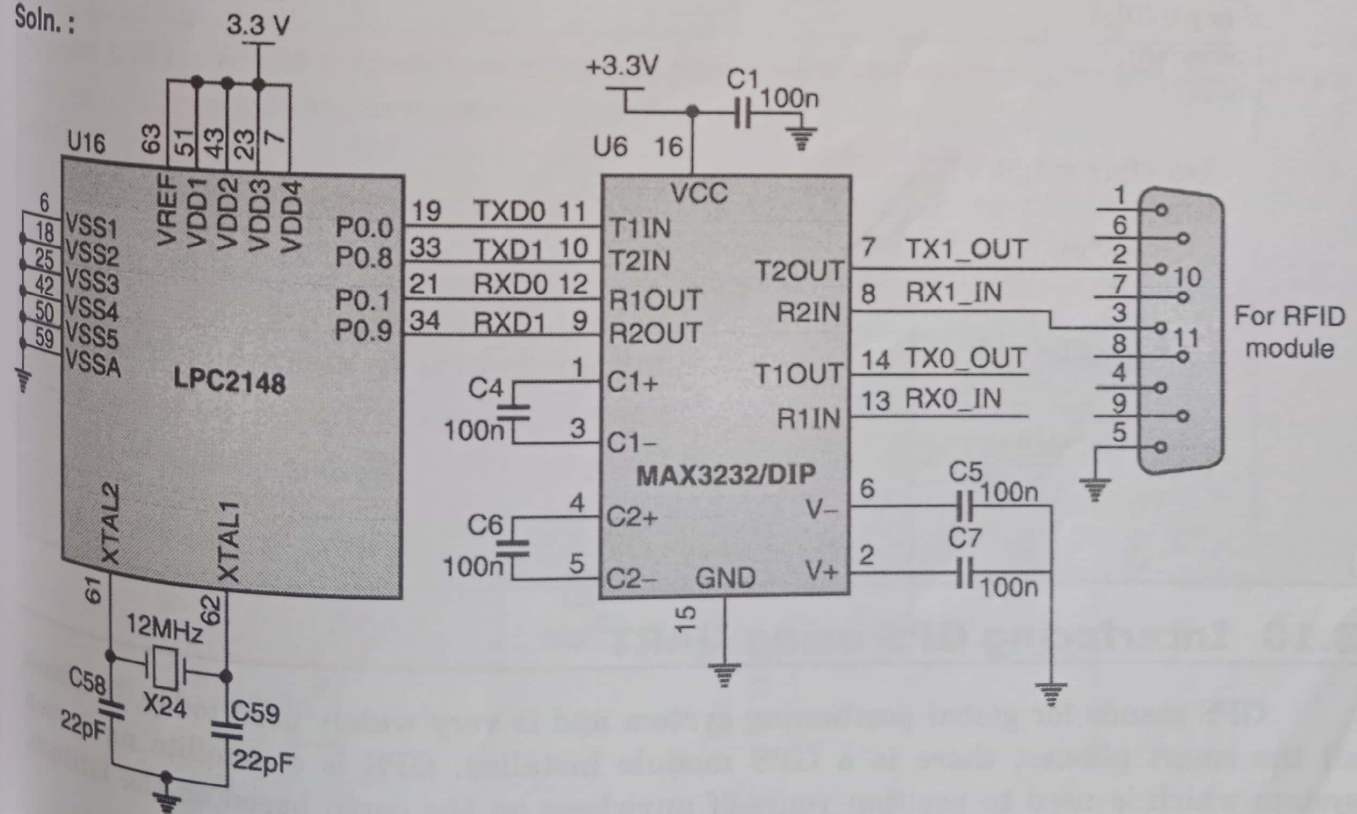


Fig. P. 3.9.1



```
#include <lpc21xx.H>
unsigned char test;
void InitSerial()
{
    PINSEL0 |= 0X00000005;           //Enabling TxD0 & RxD0
    U0LCR = 0x00000083;              //Configure in 8-bit data, no parity and 1-stop bit mode
    U0DLL = 0x00000061;              //Initialize baud rate=9600
    U0LCR = 3;
}
void send (int ch)
{
    while (!(U0LSR & 0x20));          //wait for previous transmission to complete
    U0THR = ch;                       //Give new data to be transmitted
}
char input ()
{
    while (!(U0LSR & 0x01))           //wait for data to be received
    {
    }
    return (U0RBR);                  //return the received data
}
int main()
{
    VPBDIV = 0x02;                   // Initialize for division of PCLK by two
    int i,flag=0;
    char *p=0xAA, 0xBB, 0x02, 0x01, 0x03; //checksum byte =  $02 \oplus 01 = 3$ 
    char p1[20];
    InitSerial();
    while(1)
    {
        for(i=0;i<=4;i++)
        {
            send(*p);
            p++;
        }
        for(i=0;i<=12;i++)           //13 bytes will be received as the module type is of 8 bytes
        {
            p1=input()
        }
    }
}
```

3.10 Interfacing GPS using UART ✓

GPS stands for global positioning system and is very widely used today. In almost all the smart phones, there is a GPS module installed. GPS is a satellite navigation system which is used to position yourself anywhere on the earth based on the latitude, altitude and longitude.

The interfacing of a GPS module requires simple RS232 based serial communication i.e. the RxD and TxD pins. The interfacing of a GPS module to any microcontroller can be done as shown in Fig. 3.10.1

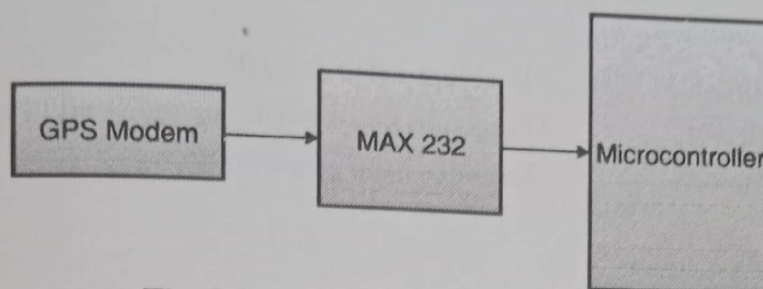


Fig. 3.10.1 : Interface of GPS module

This module is integrated with a microcontroller to help serial communication and other interfacing. It has a nine pin connector (The communication although requires only 3 pins viz. RxD, TxD and GND) with the pin connection details as shown in the Table 3.10.1.

Table 3.10.1 : GPS module pin connections

Pin No.	Symbol	IO Type	Description
1.	RXD	I	UART receiver
2.	TXD	O	UART Transmitter
9.	GND	GND	Reference

Let us see a program example to read the latitude and longitude information from the GPS module.

Program 3.10.1 : Write a program to read the data from satellite into microcontroller and also show the interfacing for the same.

Soln. :

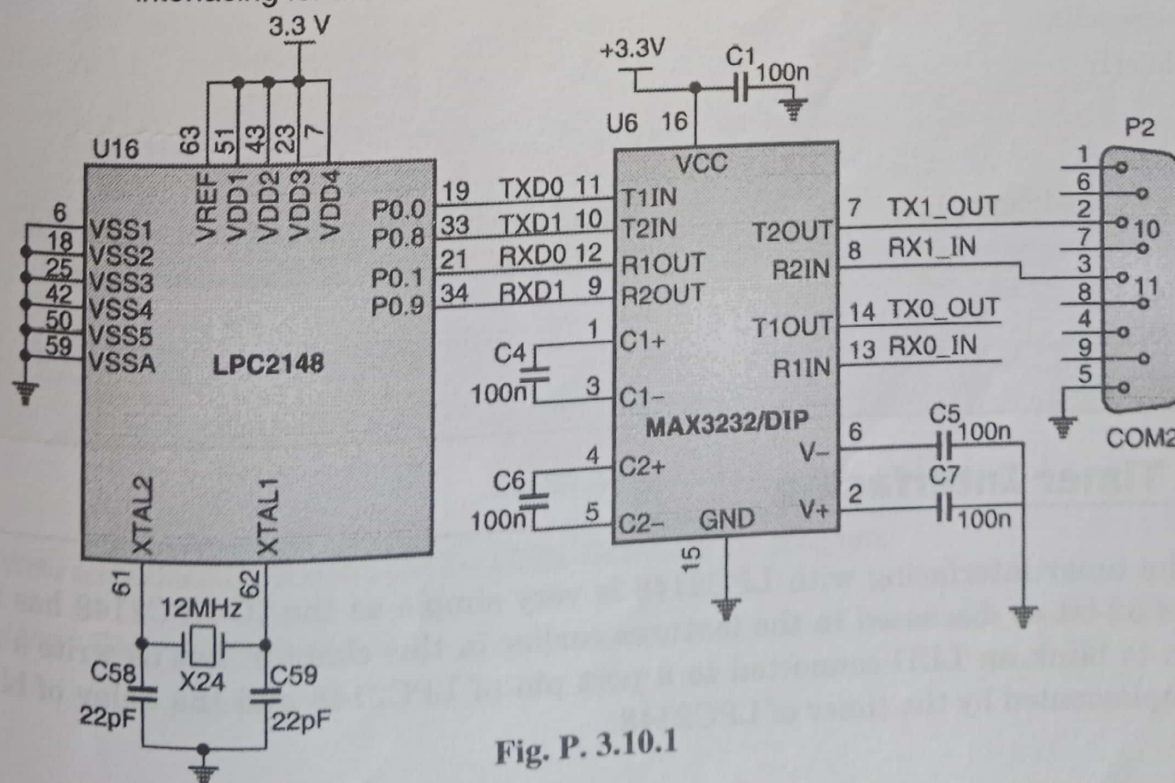


Fig. P. 3.10.1


```

#include <lpc21xx.H>
unsigned char test;

void InitSerial()
{
    PINSEL0 |= 0X00000005;    //Enabling TxDO & RxDO
    UOLCR  = 0x00000083;    //Configure in 8-bit data, no parity and 1-stop bit mode
    UODLL  = 0x00000061;    //Initialize baud rate=9600
    UOLCR = 3;
}

void send (int ch)
{
    while (!(UOLSR & 0x20));    //wait for previous transmission to complete
    if (ch=='\n') UOTHR = 0x0D;    //Give new data to be transmitted
    else UOTHR = ch;
}

char input ()
{
    while (!(UOLSR & 0x01))    //wait for data to be received
    {
    }
    return (UORBR);    //return the received data
}

int main()
{
    VPBDIV = 0x02;    // Initialize for division of PCLK by two
    InitSerial();
    while(1)
    {
        {
            send(input());    //Read the character from GPS and send it to the other
                                //serial port where the display is connected
        }
    }
}

```

3.11 Timer Interfacing

The timer interfacing with LPC2148 is very simple as the IC LPC2148 has inbuilt timers of 32-bit as discussed in the features earlier in this chapter. Let us write a simple program to blink an LED connected to a port pin of LPC2148 and the delay of blinking being implemented by the timer of LPC2148.

Program 3.11.1 : Write a program to blink LED connected on a Port 0 pins of LPC2148.

PU - April 2012

Soln. :

```
#include <lpc21xx.H>
void InitTimer0()
{
    TOCTCR = 0;
    POPR = PRESCALE - 1;
    TOTCR = 2;
}
void delay(unsigned int ms)
{
    TOTCR = 2;
    TOTCR = 1;
    while(TOTCR < ms);
    TOTCR = 0;
}
int main()
{
    VPBDIV = 0x02; // Initialize for division of PCLK by two
    InitTimer0();
    IOODIR = 0xFFFFFFFF;
    while(1)
    {
        IOOSET = 0xFFFFFFFF;
        delay(1000);
        IOOCLR = 0xFFFFFFFF;
        delay(1000);
    }
    return 0;
}
```

3.12 University Questions

April 2012

Q.1 Write embedded C code to blink the LEDs. Comment the program.

(Ans. : Refer program 3.11.1)

May 2015

- Q. 2(a) Write a program for on chip ADC for LPC 2148. (Ans. : Refer section 3.3.2) (6 Marks)
- Q. 3(a) Draw and explain interfacing diagram of I2C EEPROM to LPC 2148. (4 Marks)
- (Ans. : Refer section 3.5.1)

□□□